CMS 705
Programming Languages

Complete Exam Cheat Sheet

Rivers State University

Lecturer: the lecturer | Session: 2024/2025

Topics Covered

Language Structure · Data Types · Data Structures

Control Structures · Data Flow · Runtime

Interpretive Languages · Lexical Analysis & Parsing

1 Language Structure

💡 **A language is a formal system of symbols and rules to communicate with a computer. Every instruction must have a precise, single meaning.**

COMPONENT	WHAT IT STUDIES	CS / C++ ANALOGY
Phonology	Sound patterns — smallest units (phonemes)	`int`, `<<`, `age` — tokens
Morphology	How words/tokens are formed	`Walk + ed → Walked`; identifiers
Syntax	Rules for arranging words → sentences	`for(int i=0;i<10;i++){}`
Semantics	Meaning of words / sentences	What the code actually does
Pragmatics	Language in social / usage context	Readability, writability, reliability

🎯 **3 Key Questions a Language Must Answer:** **1. What *look* → Syntax** **2. What *mean* → Semantics** **programs** **programs**

2 Static vs Dynamic Semantics

⚙️ STATIC SEMANTICS

- Checked at **compile time**
- Type compatibility
- Variable declared before use
- Prevents execution errors

▶️ DYNAMIC SEMANTICS

- Checked at **runtime**
- How expressions are evaluated
- How values change during execution
- What actually happens when code runs

```
// STATIC SEMANTICS – compile-time error
int x;
x = "hello"; // ❌ Compile error: can't assign string to int

// DYNAMIC SEMANTICS – runtime behaviour
int c = 0;
if (flag) c = 1;   // value changes at runtime
cout << c;           // outputs: 1
```

⚠️ **Exam tip:** **Static = compile time (type errors, undeclared vars). Dynamic = runtime (actual behaviour, value changes).**

3 Types of Language — Levels of Abstraction

LEVEL	TYPE	EXAMPLE	NOTES
Lowest	Machine Language	`01001010`	Binary; CPU executes directly; no translation; machine-dependent
Low	Assembly Language	`MOV A, B`	Mnemonics; needs assembler; full hardware control
Middle	C Language	`int sum = a+b;`	Hardware access + structured; used in OS, compilers
High	Python, Java, C++	English-like syntax	Needs compiler/interpreter; machine-independent; easy to use

MIDDLE-LEVEL (C)

- Moderate portability
- Faster execution
- Manual memory management
- Systems / embedded use

HIGH-LEVEL (C++, PYTHON)

- High portability
- Slower (compilation overhead)
- Automatic memory
- Application / end-user focus

4 Implementation Methods

🔧 **Assembly**	Translates assembly language → machine code via assembler
📦 **Compilation**	Entire program translated at once → `.exe` (C, C++)
▶️ **Interpretation**	Code translated & executed **line by line** at runtime (Python, JS)
⚡ **JIT (Just-in-Time)**	Bytecode compiled to machine code **on demand at runtime** — Java / JVM

	COMPILED	INTERPRETED	JIT
Speed	Fast	Slower	Near-compiled
Output	.exe	None	Native (runtime)
Errors found	Before run	During run	Mix
Examples	C, C++	Python, JS	Java (JVM)

5 Primitive Data Types

📌 **Built-in, fixed size, stored directly in memory (CPU-level). Cannot be null. No built-in functions.**

TYPE	SIZE	EXAMPLE	USE
int	4 bytes	`int count = 10;`	Counting, indexing, loops
short int	2 bytes	`short x = 5;`	Smaller integers
long int	4–8 bytes	`long big = 99999;`	Large integers
unsigned int	4 bytes	`unsigned int u = 10;`	Non-negative only
float	4 bytes	`float t = 36.5;`	Single precision (7 digits)
double	8 bytes	`double pi = 3.14159;`	Double precision (15–16 digits)
char	1 byte	`char g = 'A';`	Single character (ASCII)
bool	1 byte	`bool ok = true;`	Decision / conditions
void	—	`void display(){}`	Functions with no return value

🧠 **int range: –2,147,483,648 to … | char uses 'A' single quotes | string uses "Alex" double quotes**

2,147,483,647

I

6 Non-Primitive Data Types

📌 Also called reference / derived types. Complex, dynamic size, have built-in functions. Built from primitive types.

STRING

```
string name = "Alex Johnson";
cout << name.length();  // 12
```

ARRAY

```
int nums[] = {10,20,30,40};
cout << nums[0];  // 10
```

STRUCT

```
struct Student {
  string name; int age;
};
```

POINTER

```
int c = 10;
int* ptr = &c;
// ptr stores address of c
```

ENUM

```
enum Day {Mon,Tue,Wed,Thu};
Day today = Wed;
cout << today; // 2
```

CLASS (OOP)

```
class Car {
public:
  string model; int year;
  void display();
};
```

	PRIMITIVE	NON-PRIMITIVE
Created by	Language itself	Programmer / library
Size	Fixed	Dynamic

Functions	None	Many built-in

7 Data Structures — Overview

💡 **A method of organising, storing, and managing data in a computer so it can be accessed and manipulated efficiently.**

LINEAR

- Sequential arrangement
- Single level
- Simple to traverse
- Array, Linked List, Stack, Queue

NON-LINEAR

- Hierarchical / graph-like
- Multiple levels
- Complex to traverse
- Tree, Graph, Heap, Hash Table, Trie

OPERATION	DESCRIPTION
Insertion	Add element (beginning, end, or position)
Deletion	Remove element + free memory
Traversal	Visit ALL elements once — "go through everything"
Searching	Find specific element — "stop when found" (Linear or Binary)
Updating	Change value of existing element
Sorting	Arrange ascending/descending (Bubble, Selection, Insertion)
Merging	Combine two data structures into one

⚠️ **Traversal vs Searching:** **Traversal visits every element. Searching stops as soon as it finds the target.**

8 Linear & Non-Linear Structures

STACK — LIFO

Push (insert) | Pop (remove)

Function calls · Undo/redo · Browser back

QUEUE — FIFO

Enqueue (rear) | Dequeue (front)

Printer queue · CPU scheduling

ARRAY

Contiguous memory, fixed size, O(1) access by index

BINARY TREE

Each node ≤ 2 children. Left < Parent < Right (BST)

HASH TABLE

key → hash function → index. O(1) avg. lookup/insert/delete

HEAP

Max Heap: Parent ≥ Children
Min Heap: Parent ≤ Children

```
// Singly Linked List — key exam implementation
struct Node { int data; Node* next; };
Node* head = NULL;
void insertEnd(int value) {
  Node* newNode = new Node();
  newNode->data = value;  newNode->next = NULL;
  if (head == NULL) { head = newNode; return; }
  Node* temp = head;
  while (temp->next != NULL) temp = temp->next;
```

```
    temp->next = newNode;
}
```

9 Control Structures

💡 **Determine the order of execution — which statement runs, when, and how many times.**

#	TYPE	PURPOSE	KEYWORDS
1	Sequential	Default top-to-bottom	(none — default)
2	Selection	Branch on condition	`if`, `else`, `else if`, `switch`
3	Iteration	Repeat a block	`for`, `while`, `do-while`, `for-each`
4	Jump	Transfer control unconditionally	`break`, `continue`, `return`, `goto`

```
// For loop (known iterations)
for (int i=1; i<=5; i++)
  cout << i;

// While (condition-based)
int i = 1;
while (i <= 5) { cout << i; i++; }

// Do-While (runs at least ONCE)
do { cout << i; i++; }
while (i <= 5);
```

```
// For-each
int nums[] = {1,2,3,4,5};
for (int n : nums) cout << n;

// Break vs Continue
if (i==5) break;    // EXIT loop
if (i==6) continue; // SKIP iteration

// Switch
switch(day){
  case 1: cout<<"Mon"; break;
  default: cout<<"Other";
}
```

10 Data Flow

💡 **The movement, transformation, and usage of data within a program: Input → Storage → Processing → Output**

Input → **Storage** → **Processing** → **Output**

BY PROGRAM EXECUTION	
Sequential	Line by line: A→B→C→D
Conditional	Branches on condition (if/else)
Iterative	Data flows repeatedly through loops
Procedural	Data moves between functions via parameters/return

EXPLICIT FLOW

- Clearly written in code
- Passed as parameters
- Returned from functions
- Easy to trace & debug

IMPLICIT FLOW

- Through global variables
- Hidden / indirect
- Harder to trace
- Can cause side effects

11 Runtime Considerations

💡 **Runtime = the phase when a compiled/interpreted program is actually executing on the CPU.**

📦 **Stack**	Local variables, function calls I **LIFO**, auto-freed when function ends
🌐 **Heap**	Dynamic allocations via `new` I Must manually free with `delete`
🌍 **Global**	Global / static variables I Lives for entire program lifetime
📜 **Code**	Compiled instructions (text segment) — read-only

```
int globalCounter = 100;    // Global area
int processData(int value) {
  int localVar = value * 2;   // Stack — auto freed
  int* heapVar = new int(5);  // Heap — must free!
  int result = *heapVar;
  delete heapVar;              // Free heap memory
  return result;
}
```

	COMPILE TIME	RUNTIME
When	Before execution	During execution
Errors	Syntax, type errors	Logic, null pointer, division by zero
Semantics	Static	Dynamic

12 Interpretive Languages

	COMPILED	INTERPRETED	JIT
Translation	Whole program at once	Line by line at runtime	Bytecode → machine code on demand
Output file	.exe produced	No output file	Native code at runtime
Speed	Fast (pre-translated)	Slower	Near-compiled
Portability	Lower (platform-specific)	High	High (via JVM)
Examples	C, C++	Python, JavaScript	Java (JVM)

HOW INTERPRETATION WORKS

📄 Source Code (.py)

↓

🔍 Read line 1 → Translate → Execute

↓

🔍 Read line 2 → Translate → Execute

↓

❌ Error on line 3 → Stop

HOW JIT WORKS (JAVA)

📄 Java Source (.java)

↓ javac compiler

⚙️ Bytecode (.class)

↓ JVM

⚡ JIT → Machine Code

↓

🚀 Fast Execution

⚠ Interpreter never produces a saved executable. JIT produces machine code but only at runtime — not saved to disk.

13 Lexical Analysis & Parsing

📖 **Source Code**	Raw text written by programmer
🔡 **Lexical Analysis**	Groups characters → **TOKENS** (keywords, identifiers, operators, literals)
🌳 **Parsing**	Checks grammar rules → builds **Parse Tree / AST**
🧠 **Semantic Analysis**	Checks meaning — types, declarations, compatibility
⚙️ **Code Generation**	Produces machine code / bytecode

```
// Source: int age = 20;
// Tokens: [KEYWORD:int] [IDENT:age] [OP:=] [LIT:20] [DELIM:;]

// Lexical Error (invalid token):
int @age = 5;    // ❌ '@' not valid token
// Syntax Error (valid tokens, wrong grammar):
int = age 5;     // ❌ wrong arrangement
// Semantic Error (valid syntax, wrong meaning):
int age = "hello"; // ❌ type mismatch
```

LINGUISTICS	PROGRAMMING LANGUAGE	COMPILER STAGE
Phonology	Characters, symbols	Character scanning
Morphology	Tokens — keywords, identifiers	Lexical Analysis
Syntax	Valid statements & expressions	Parsing
Semantics	Meaning of code	Semantic Analysis

Quick Reference

KEY DEFINITIONS	
Syntax	Rules for valid code structure
Semantics	What the code means
Pragmatics	Readability, writability, reliability
Data Type	What a variable holds + ops allowed
Parameter	Variable declared in function definition
Argument	Value passed when calling a function
Stack	LIFO — local vars, auto-freed
Heap	Dynamic alloc, manual `delete`

🎯 Exam Tips

- **Know Singly Linked List C++ code**
- **Know Binary Tree C++ code**
- **Know Grade Calculator (procedural data flow)**
- **break = exits loop; continue = skips iteration**
- **Static = compile time; Dynamic = runtime**
- **do-while runs at least ONCE**
- **JIT: bytecode → machine code at runtime**
- **Phonology→Morphology→Syntax = Lexer→Parser**

📋 5 Course Topics

1. **Language Definition Structure**
2. **Data Types & Structures**
3. **Runtime Considerations**
4. **Interpretive Languages**
5. **Lexical Analysis & Parsing**

Traversal	Visit every element once
Searching	Stop when element found